

DeepAgents 实战：多 Agent 架构的深度调研助手 已付费

原创 神说要有光 神光的幸福生活 2026年5月29日 22:30 山东

前面学了 DeepAgents 的各种 middleware，但是太散了。

而且 middleware 里带的 tool 需要在 prompt 里说明怎么用。

那有没有一个整合所有中间件的 api，并且内置了 prompt 呢？

有的，就是 createDeepAgent

我们基于 deepagents 开发一个多 Agent 项目：深度调研助手

你只要给它一个主题，它的主 Agent 会自动规划任务，列出 todo 列表，然后交给不同的子 Agent 来执行任务，比如联网搜索、代码执行等，最后生成一份调研报告

我们分了三个子 Agent：

主 Agent：整个系统的编排中心，负责把用户输入的调研主题拆解成可执行流程，并协调各子 Agent 分工完成。它不亲自包揽所有调研细节，而是按「规划 → 调研 → 分析 → 起草 → 审阅 → 定稿」推进任务：先用待办列表明确步骤，再按需委派子 Agent，最后由自己整合材料、撰写报告并根据编辑反馈修订定稿。

调研员子 Agent (researcher)：每次只负责一个聚焦的子主题。通过联网搜索收集资料，将关键事实与来源 URL 整理成结构化摘要，写入 findings_*.md。多个调研员可并行工作，适合把大主题拆成若干子方向同时推进。

分析师子 Agent (analyst)：当调研涉及数字对比、排名、增长率等计算时启用。在 QuickJS REPL 中执行 JavaScript 完成数值分析，禁止凭猜测给出数字。结果写入 analysis_*.md，供主 Agent 写报告时引用。

编辑子 Agent (editor) 在报告草稿完成后介入，从准确性、结构完整性、来源引用、语言表述等维度审阅，返回具体修改建议。编辑不直接改写报告，审阅与修订分离，便于主 Agent 在保持整体思路的前提下做针对性修改。

大概有这个 4 个 Agent

创建项目：

```
mkdir deep-research-assistant
cd deep-research-assistant
npm init -y
```

进入项目，安装依赖：

```
pnpm install @langchain/core @langchain/langgraph @langchain/openai @langchain/quickjs de
```

写一下 .env 配置：

```
OPENAI_API_KEY=sk-xx
OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1"
```

```
OPENAI_MODEL="qwen-plus"

# 用于身份验证, 实现链路上报
LANGCHAIN_API_KEY=xxx

# 指定LangSmith中的项目, 追踪结果会归类到该项目下
LANGCHAIN_PROJECT=deep-research-assistant

# 开启LangSmith追踪功能
LANGCHAIN_TRACING_V2=true

BOCHA_API_KEY=sk-xx
```

开启 langsmith 追踪

这里用到网络搜索, 配置下博查的 api key

然后创建 src/tools/search.mjs

```
import { tool } from "langchain";
import { z } from "zod";

const BOCHA_API_URL = "https://api.bochaai.com/v1/web-search";

function formatWebPages(webpages) {
  return webpages
    .map(
      (page, idx) =>
        `引用: ${idx + 1}
标题: ${page.name ?? ""}
URL: ${page.url ?? ""}
摘要: ${page.summary ?? ""}
网站名称: ${page.siteName ?? ""}
网站图标: ${page.siteIcon ?? ""}
发布时间: ${page.dateLastCrawled ?? ""}`,
    )
    .join("\n\n");
}

async function bochaWebSearch(query, count) {
  const apiKey = process.env.BOCHA_API_KEY?.trim();
  if (!apiKey) {
    return "Bocha 联网搜索的 API Key 未配置 (环境变量 BOCHA_API_KEY), 请先在 .env 中配置后再重试。"
  }

  const response = await fetch(BOCHA_API_URL, {
    method: "POST",
    headers: {
```

```

    Authorization: `Bearer ${apiKey}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    query,
    freshness: "noLimit",
    summary: true,
    count,
  }),
});

if (!response.ok) {
  const errorText = await response.text();
  return `搜索 API 请求失败, 状态码: ${response.status}, 错误信息: ${errorText}`;
}

let json;
try {
  json = await response.json();
} catch (e) {
  return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e.message}`;
}

try {
  if (json.code !== 200 || !json.data) {
    return `搜索 API 请求失败, 原因是: ${json.msg ?? "未知错误"}`;
  }

  const webpages = json.data.webPages?.value ?? [];
  if (!webpages.length) {
    return `未找到与「${query}」相关的结果。`;
  }

  return formatWebPages(webpages);
} catch (e) {
  return `搜索 API 请求失败, 原因是: 搜索结果解析失败 ${e.message}`;
}
}

export const webSearch = tool(
  async (input) => {
    const count = input.count ?? 10;
    console.log(`🔍 搜索: ${input.query} (${count} 条)`);
    return bochaWebSearch(input.query, count);
  },
  {
    name: "web_search",
    description:

```

```

    "使用 Bocha 联网搜索 API 检索互联网网页。输入中文或中英结合的搜索关键词，可选 count 指定结果数量
schema: z.object({
  query: z
    .string()
    .min(1)
    .describe("搜索关键词，优先使用中文，例如：2026年 AI Agent 框架对比、LangGraph 最新动态"),
  count: z
    .number()
    .int()
    .min(1)
    .max(20)
    .optional()
    .describe("返回的搜索结果数量，默认 10 条"),
}),
},
);

```

这个就是网络搜索的 tool

然后写下 4 个 Agent:

src/agent.mjs

```

import path from "node:path";
import { fileURLToPath } from "node:url";
import dedent from "dedent";
import { ChatOpenAI } from "@langchain/openai";
import { createCodeInterpreterMiddleware } from "@langchain/quickjs";
import { createDeepAgent, FilesystemBackend } from "deepagents";

import { webSearch } from "../tools/search.mjs";

const projectDir = path.resolve(
  path.dirname(fileURLToPath(import.meta.url)),
  "..",
);

const researcherSubAgent = {
  name: "researcher",
  description:
    "通过联网搜索调研单一子主题。每次只分配一个子主题；多个独立子主题可并行启动多个调研员。",
  systemPrompt: dedent`
    你是一名专业调研员，负责调研**一个**分配给你的子主题，并写入**一份**调研结果文件。

    ## 工作流程（严格遵守，禁止空转循环）
  `
};

```

1. ****可选****: 用 `write_todos` 列出最多 3 条中文执行步骤（例如「搜索官方文档」「搜索社区评价」「整理并」
2. 最多调用 3 次 `web_search`（硬性上限，绝不超过）
3. 将搜索结果整理为结构化摘要，包含关键事实与来源 URL
4. 调用 `write_file` ****一次****，保存到任务指定的路径（必须在 `/workspace/sources/findings_*.md`）
5. 用一句话确认已完成，然后****立即停止****，不要再搜索、写文件或更新 `todo`

`write_todos` 使用规则（若使用）

- 最多 3 条，每条 `content` 必须用中文
- 仅用于拆解本子的调研步骤，不要重复主 Agent 已完成的总体规划
- 最后一条 `todo` 必须是「写入 `findings` 文件」；该步骤完成后将所有 `todo` 标为 `completed` 并结束

其他规则

- 不要重复相同的搜索关键词
- `write_file` 完成后禁止再次搜索——你的任务已结束
- 其他人只能看到你写入的文件，内容必须完整、自治
- ****所有输出必须使用中文****（专有名词如 `LangGraph` 可保留英文）
- 搜索关键词优先使用中文；若主题本身是英文专有名词，可中英结合

```
\n,\ntools: [webSearch],\n};\n\nconst editorSubAgent = {\n  name: "editor",\n  description:\n    "审阅报告草稿的准确性、结构与完整性。在 /workspace/reports/draft_*.md 写好后使用。",\n  systemPrompt: dedent`\n    你是一名资深情报编辑，负责**审阅**报告草稿——**不要**亲自改写报告。`,\n};
```

阅读材料

- 原始问题: `/workspace/sources/question.txt`
- 待审草稿: 任务中指定的路径
- 支撑材料: `/workspace/sources/` 下的调研文件（如需要）

审阅要点

- 报告是否直接回答了原始问题？
- 章节结构是否清晰，段落是否充实（而非只有 `bullet` 列表）？
- 是否引用了来源，并在「参考资料」章节列出？
- 是否有遗漏、无依据的断言或缺失的视角？
- 语言是否为中文，表述是否专业？

输出

返回简洁的审阅意见和具体、可操作的修改建议。

****不要****写入报告文件，只提供反馈。所有输出使用中文。

```
\n,\n};\n\nconst analystSubAgent = {\n  name: "analyst",\n  description:\n    "使用 eval REPL 进行数值计算与结构化数据分析。适用于计算、排名、同比对比或 JSON/CSV 分析。",\n  systemPrompt: dedent`\n    你是一名数据分析师，所有计算必须通过 eval REPL 完成--**禁止**猜测数字。\n\n    ## 工作流程\n\n    1. 从 /workspace/sources/ 读取数据文件（或从调研结果中提取数字）\n    2. 在 REPL 中编写并运行 JavaScript，计算总和、均值、排名、增长率等\n    3. 将分析结果保存到 /workspace/sources/analysis_*.md，包含计算逻辑与结论\n\n    必须展示计算过程，结论可从 REPL 输出复现。所有输出使用中文。`,\n  middleware: [createCodeInterpreterMiddleware()],\n};\n\nconst orchestratorPrompt = dedent`\n  你是「深度调研助手」的主 Agent，负责协调调研、分析与编辑，产出高质量调研简报.\n\n  ## 语言要求\n\n  - **所有输出必须使用中文**：对话回复、write_todos 任务列表、文件内容、搜索关键词\n  - write_todos 中每条 todo 的 content 必须用中文描述，例如「撰写调研计划」「委派调研员调研 LangGrap\n  - 搜索时优先使用中文关键词；英文专有名词（如 LangGraph、AutoGen）可保留\n  - 报告、调研笔记、计划文件全部用中文撰写\n\n  ## 你的职责\n\n  协调调研员、分析师和编辑完成报告。不要亲自完成所有调研--将专业工作委派给子 Agent.\n\n  ## 标准流程\n\n  1. **规划** - 用 write_todos 拆解任务（中文）。将用户问题保存到 /workspace/sources/question.txt\n  2. **调研** - 按 web-research 技能：写 research_plan.md，委派调研员子 Agent（可并行）\n  3. **分析** - 若涉及数字对比或数据表，委派分析师子 Agent\n  4. **起草** - **由你亲自**按 report-writer 技能撰写，用 write_file 写入 /workspace/reports/d\n  5. **审阅** - 委派编辑子 Agent 审稿，根据反馈修订一次\n  6. **定稿** - 保存最终报告到 /workspace/reports/report_[主题]_[日期].md`
```

```

## task 工具 (子 Agent 委派)

**仅**以下 subagent_type 合法: researcher、analyst、editor、general-purpose。

- web-research、report-writer 是**技能** (写作指南), **不是**子 Agent, 禁止作为 subagent_type
- 报告起草、修订、定稿由**主 Agent 自己**用 write_file / edit_file 完成, 不要委派 task

## 委派规则

- 每个调研员只负责一个聚焦的子主题
- **每份报告最多 3 个调研员**——只选最相关的子主题
- 框架对比类任务: 优先调研用户明确点名的框架; 否则选最重要的 3 个
- 最多并行启动 3 个调研员, 已有 3 份 findings 文件后不再新增调研员
- 仅在确实需要数值计算时使用分析师
- 每份报告只调用编辑一次 (草稿完成后)
- 调研完成后直接进入起草 → 审阅 → 定稿, 不要额外开调研轮次

## 文件约定

- 计划与原始资料: /workspace/sources/
- 草稿与终稿: /workspace/reports/
- 同一时间只编辑一个文件, 避免冲突

## 完成时告知用户

- 最终报告保存路径
- 2-3 句话的核心发现摘要
- 调研中的局限或信息缺口
`;

exportfunction createIntelligenceDeskAgent() {
const apiKey = process.env.OPENAI_API_KEY?.trim();
if (!apiKey) {
    thrownewError("未设置 OPENAI_API_KEY 环境变量");
}

const model = process.env.OPENAI_MODEL?.trim() || "gpt-4o";
const baseUrl = process.env.OPENAI_BASE_URL?.trim() || undefined;

const backend = new FilesystemBackend({
    rootDir: projectDir,
    virtualMode: true,
});

return createDeepAgent({
    model: new ChatOpenAI({
        model,

```



```

    temperature: 0,
    apiKey,
    ...(baseUrl
      ? {
          configuration: {
            baseUrl,
          },
        }
      : {}),
  }),
  systemPrompt: orchestratorPrompt,
  backend,
  memory: [path.join(projectDir, "AGENTS.md")],
  skills: ["/skills/"],
  subagents: [researcherSubAgent, editorSubAgent, analystSubAgent],
});

export { projectDir };

```

我们直接用 createDeepAgent 的 api，这样不用自己组装 middleware 了，配置下 skills 目录、memory 的文件路径、子 agent 就好了。

这里的 dedent 是去掉换行和缩进的空格，换成 \n 的：

写代码的时候正常缩进，用这个可以自动去掉换成 \n

然后分析的子 Agent 需要执行代码，用到了 quickjs 这个 js 引擎来执行：

还有一个 `src/cli.mjs` 就是调用这个 agent，格式化下输出（直接从仓库复制吧）

还有两个 skill：

skill 就是对 prompt 的封装，这里就是告诉 agent 怎么网络搜索、怎么写报告的

跑一下：

```
node src/cli.mjs "调研国家统计局公开的2023年省级地区生产总值（GDP）数据：提取GDP总量前6名省份的具体数
```

神光的编程秘籍

这样，一个可以加载 skills、有自动读取长期记忆 Agents.md、多个子 Agent 的 Agent 就完成了。

网络搜索的 tool 前面用过几次了。

重点是这个沙箱执行代码的 tool

大模型不会数学计算，涉及到计算的都是生成代码，用 eval 的 tool 来执行，这里是 js 代码用 quickjs 引擎来执行。

当然，你生成别的语言的代码也行，用对应的引擎执行即可。

这里有个 langsmith 小技巧：

可以通过 filter 过滤出所有的 tool 来，更容易理清流程。

最后就是 todo 了，复杂任务不能走一步想一步，都要提前生成 todo 列表，一步步执行：

主 agent 的 todo 列表：

当前的是 in_progress，完成后会标记为 completed

这个是调研员子 Agent 的 todo 列表：

其实这个中间件是 langchain 提供的，我们用一下试试：

src/todo-middleware-test.mjs

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";
import {
  createAgent,
  HumanMessage,
  todoListMiddleware,
} from "langchain";

const model = new ChatOpenAI({
  model: process.env.OPENAI_MODEL,
  apiKey: process.env.OPENAI_API_KEY,
  temperature: 0,
  configuration: {
    baseUrl: process.env.OPENAI_BASE_URL
  }
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "你是生活规划助手。收到需要多步完成的请求时，先用 write_todos 列出中文执行步骤，然后简要说明你的计划",
  middleware: [todoListMiddleware()],
});

const query =
  "我下周末想带爸妈去杭州玩两天，帮我规划一下：交通怎么选、住哪里方便、必去景点和吃什么，预算控制在人均 1500";

const result = await agent.invoke({
  messages: [new HumanMessage(query)],
});

console.log("todos:", JSON.stringify(result.todos, null, 2));
console.log("-".repeat(50));
console.log("回复:", result.messages.at(-1)?.content);
```

用 createAgent + todoListMiddleware

这个中间件自带了 write_todos 的 tool，会生成 todo 列表写到 graph 的 state 里

神光的编程秘籍

这是 langchain 提供的中间件：

我们的 Agent 里也可以用。

通过这个 agent，我们把 deepagents 的 createDeepAgent 的 todo 规划、多 Agent 执行、skill、memory 等用了一遍。

相比自己调用中间件，它集成了各种中间件，内置了对应的 prompt，用起来更简单

上下文压缩这块，内置逻辑是每个模型有输入上下文限制，达到 85% 会触发总结，保留 10%

qwen 模型没这个，我们我们可以这样改：

src/max-input-tokens-test.mjs

```
import "dotenv/config";
import { ChatOpenAI } from "@langchain/openai";

const model = new ChatOpenAI({
```

```
    model: process.env.OPENAI_MODEL,  
    apiKey: process.env.OPENAI_API_KEY,  
    temperature: 0,  
    configuration: {  
      baseUrl: process.env.OPENAI_BASE_URL  
    }  
  });  
  
console.log(model.profile.maxInputTokens);  
  
Object.defineProperty(model, "profile", {  
  get: () => ({ maxInputTokens: 131_072 }),  
});  
  
console.log(model.profile.maxInputTokens);
```

神光的编程秘籍

我们改下这个值，就可以实现对上下文压缩触发阈值的修改：

而且，触发摘要后，会把会话原文记录在 conversation_history 目录下归档：

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们基于 DeepAgents 的 createDeepAgent api 实现了深度调研助手。

这是一个多 Agent 架构的 agent

主 Agent 会列出 todo 列表，按步执行，具体的调研、数据计算分析、报告编辑，由三个子 Agent 负责，它们有各自的能力，比如网络搜索、沙盒执行代码

子 Agent 执行的时候，如果需要多个步骤，也是先列 todo 列表再执行，比如调研的时候。执行完更改 todo 任务状态。

Agents.md 的长期记忆、skill 执行 等都是内置了，配置一下就行。

上下文压缩也是内置功能，可以修改 profile.maxInputTokens 来修改触发阈值。

这样，我们没有写很多代码，就完成了了一个多 Agent 架构支持 skill 的功能比较完善的 Agent，这就是 DeepAgents 开发 Agent 的好处，有很多开箱即用的能力。

2774 人付费



上一篇

DeepAgents：开箱即用的 skill、上下文压缩等 middleware

下一篇

PostgreSQL：AI 时代最适合的数据库

修改于2026年6月1日

留言 10

写留言  

 芝诺Zenos  安徽 5月29日

这个实战很棒，希望后期多一点小实战，少一点API堆砌

首评

1

 神光的幸福生活 作者 5月29日

底层 api 肯定是要讲的，没有那些中间件，就没有这些上层的开箱即用的功能

2

 princess 浙江 5天前

怎么写出来那些规范的systemPrompt的

 神光的幸福生活 作者 5天前

ai 写的，告诉agent。职责就行

 princess 浙江 4天前

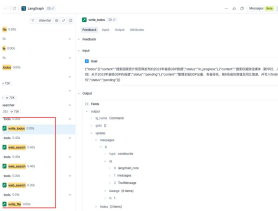
回复 神光的幸福生活: 那skill.md这个文件也是ai写的吗

 神光的幸福生活 作者 4天前

回复 princess: 是的

 鹏涛 北京 6月3日

@神光的幸福生活 光神，为啥 langSmith 中的 web_search 只有一个开始 write_todos,直到执行完了也没有去调用更改这个todo



 神光的幸福生活 作者 6月3日

你看下后面 todo 状态改了没

 谁人知 山东 5月30日

参数这些都没有类型吗？没类型不是容易搞错嘛

 神光的幸福生活 作者 5月30日

有啊，这里没用 ts，可以迁移到 ts

